

Vela

Confidential Execution on Public Chains

Technical Masterclass

Vela Acceleration Season

[Paolo Cappelletti] · [Engineering manager & Horizen Vela Team lead] · 15 September 2026

Agenda



Code walkthrough on the vela-nova example app

<https://github.com/HorizenOfficial/vela-nova>

Focus on these topics:

- key handling:
 - why we need an additional P521 key, how it is mapped to an ethereum address, who handles it
- application data vs application state
- mandatory methods to be implemented
- type of standard requests
- response format

High level architecture

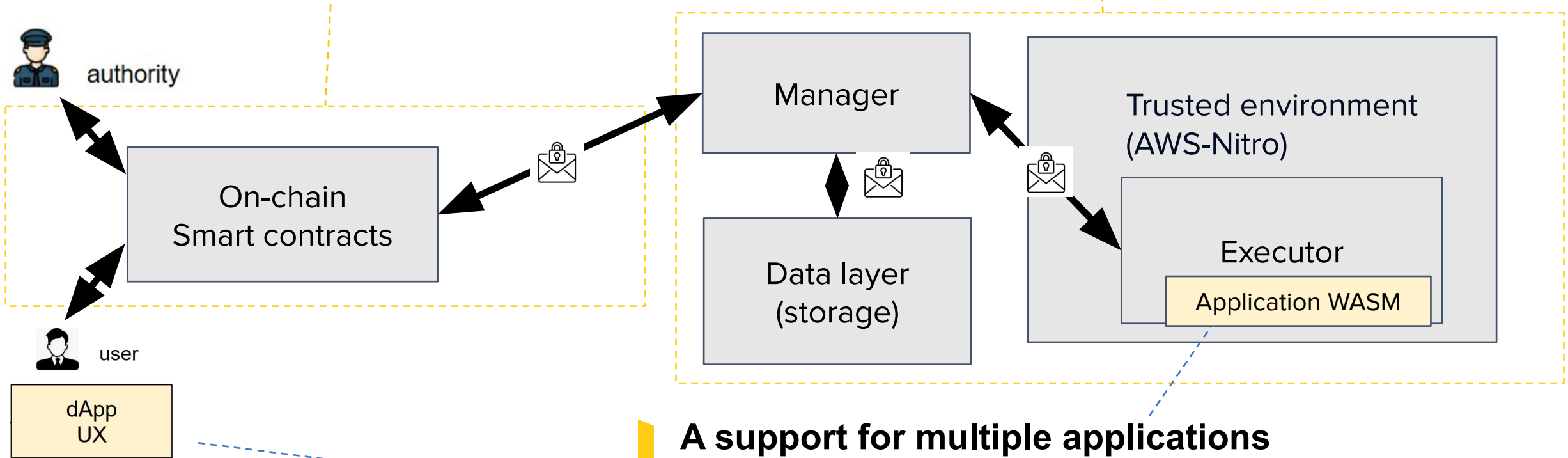


A set of smart contracts

- entry point for deanonymizations and actions requests and deposits
- exit points for results (withdrawals)
- Source for events (listened by wallets)

An off-chain execution engine

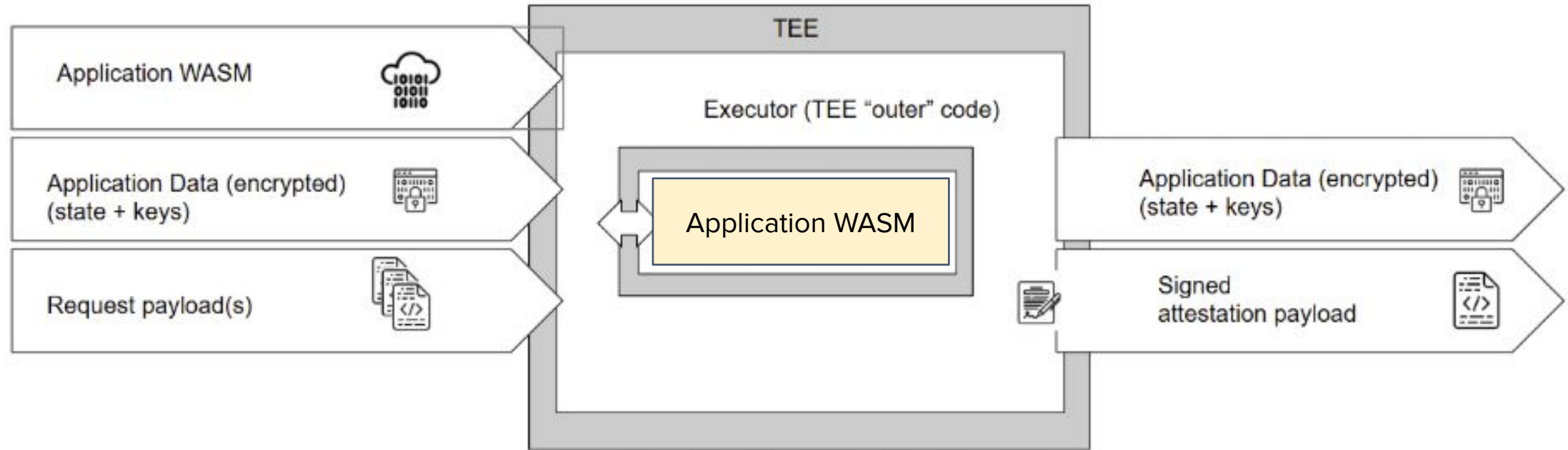
- Data is decrypted and processed only inside an isolated trusted execution environment



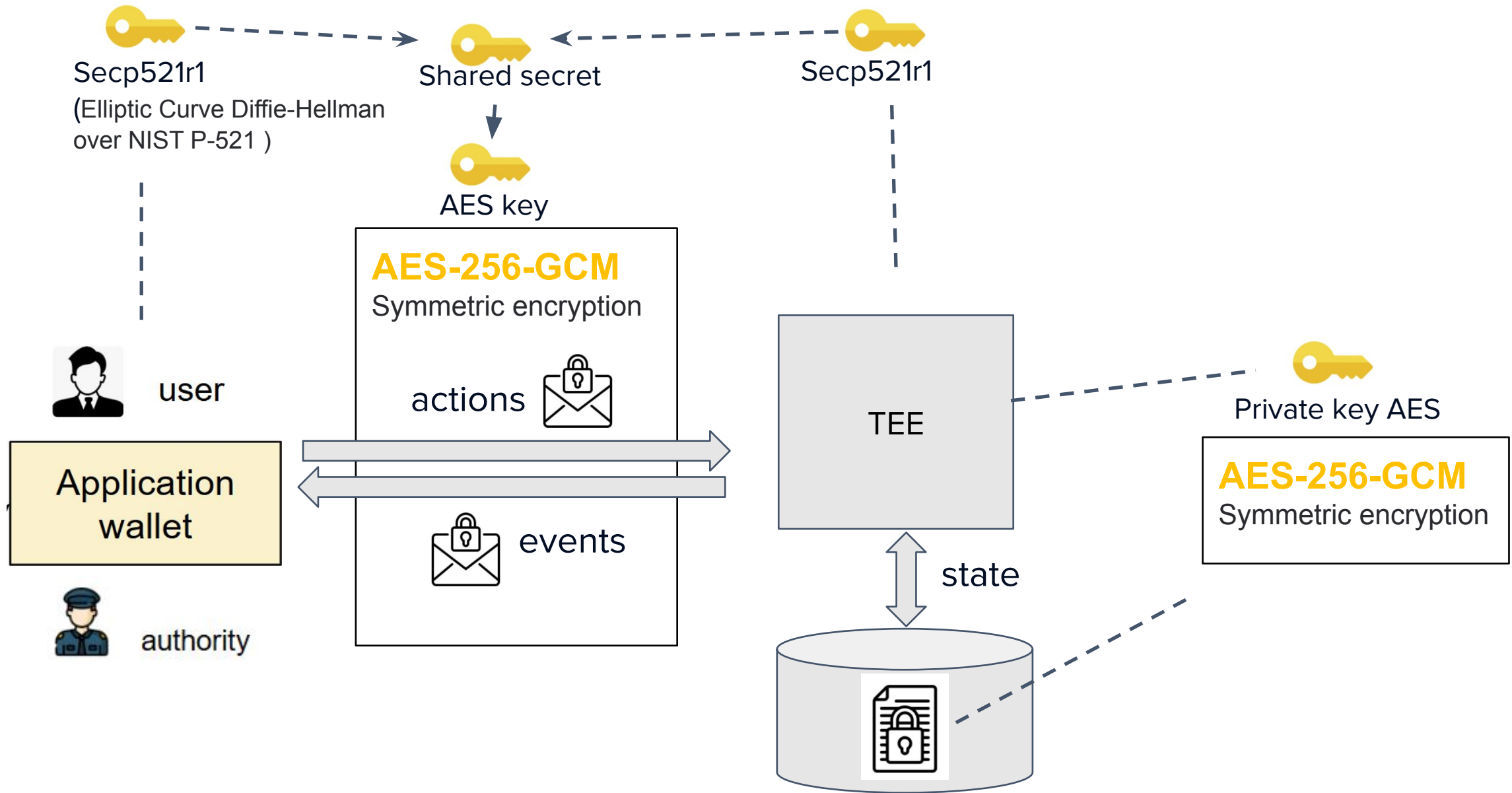
A support for multiple applications

- Possibility to serve different use-cases by developing a wallet (for the frontend part), and a wasm (for the application logic)

What runs inside the Trusted Environment



How data is encrypted



The AppData structure

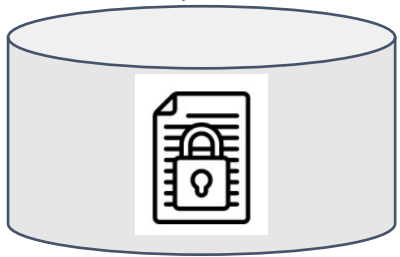


```
version      uint8
appNonce     uint64
wasmFingerprint [32]byte
appKeys      map[ethCommon.Address]*cryptotypes.PublicKeyP521
appEventSeeds map[ethCommon.Address][65]byte
appState     []byte
```

managed by the Executor

managed by your app

encrypted and saved offchain



SHA256

Root published onchain
on every state update

Methods to be implemented by the WASM



```
// called whenever the WASM is loaded in memory
// (deprecated -> use the Deploy one for initialization)
func load_module(appId int64) *byte

// Called one time during the deploy, returns the initial state
func deploy(appId int64, paramsPtr *byte, paramsLen int32) *byte

// Processes a deposit with token awareness (tokenAddress = 0x0 for ETH)
func deposit(appId int64, senderPtr *byte, senderLen int32, tokenPtr *byte, tokenLen int32,
            valuePtr *byte, valueLen int32, statePtr *byte, stateLen int32) *byte

//Processes a request and returns the new state, events, withdrawals,
// and optionally a deanonymization report
func process_request(appId int64, senderPtr *byte, senderLen int32, requestType int32,
                    payloadPtr *byte, payloadLen int32, statePtr *byte, stateLen int32) *byte
```

The input: type of requests



Type	Payload content	Reaches the WASM
DEPLOY	ArtifactID, wasm fingerprint, constructor params (unencrypted)	
ASSOCIATEKEY	Public key P521 (unencrypted) (optional) event seed (encrypted)	
DEANONYMIZE	Encrypted	
PROCESS	Encrypted	
TRUSTPROCESS	Unencrypted	

The output: process result



```
type ProcessResult struct {  
    State      []byte  
    Events     []PlainEvent  
    AppEvents  []AppEvent  
    Withdrawals []Withdrawal  
    Report     []byte          // Optional deanonymization report  
    Fuel       *Uint256  
    Error      string  
}
```

The output: type of events and withdrawals



Type	Content		
Event	UserID (recipient Address) EventSubtype Data		must be in the key registry (previous ASSOCIATEKEY) otherwise => error the Executor adds here a privacy-preserving subtype if a seed is registered for this user, otherwise the wasm app can specify an arbitrary subtype the Executor encrypts this for you
ApplicationEvent	EventSubtype Data		not touched by the Executor (copy as is)
Withdrawal	TokenAddress // 0x00 for ETH DestinationAddress Amount		